

DESIGN AND ANALYSIS OF ALGORITHMS

ALGORITHM RACE DASHBOARD

Comparing sorting and searching performance on
identical datasets

MERGE SORT

QUICK SORT

HEAP SORT

BINARY SEARCH

Python | Streamlit | Plotly

Problem Statement

Purpose of the prototype

- Compare algorithms using identical datasets.
- Show important algorithm decisions and intermediate steps.
- Measure execution time and operation counts.
- Run Binary Search on sorted data and show search steps.
- Explain why measured performance can differ from theoretical complexity.

Goal: make DAA concepts observable, measurable, and easy to understand.

Core Algorithms

Algorithms included in the prototype

MERGE SORT | $O(n \log n)$

Divide into halves, recursively sort, then merge. Predictable performance; uses extra memory.

QUICK SORT | $O(n \log n)$ AVG

Partition around a pivot and recursively sort. Pivot choice affects performance; worst case $O(n^2)$.

HEAP SORT | $O(n \log n)$

Build a heap and repeatedly extract the maximum. In-place with low auxiliary memory.

BINARY SEARCH | $O(\log n)$

Works on sorted data and repeatedly halves the search interval.

Datasets

Controlled input for fair comparison

Random

Unordered integers

Typical unsorted input

Sorted

Ascending values

Already ordered input

**Reverse
sorted**

Descending values

Tests pivot-sensitive cases

Nearly sorted

Few displaced values

Tests small disorder

Duplicates

Repeated values

Tests equal-key behavior

Suggested sizes: 100, 1,000, 5,000 and 10,000 elements. Use the same original array for every sorter.

System Workflow

From dataset generation to explanation



Binary Search path: sorted data → choose target → midpoint comparison → shrink interval → found/not found.

Performance Metrics

What the dashboard measures

Execution Time

Elapsed time for each run.

Comparisons

Element comparisons or target checks.

Swaps / Writes

Data movement and assignments.

Recursion Depth

Maximum active recursion depth.

Memory

Additional algorithm memory.

Correctness

Verify output against expected result.

Complexity & Interpretation

Reference for understanding measured behavior

Algorithm	Best	Average	Worst
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

- Actual time is affected by input pattern, pivot choice, memory behavior, Python overhead, and system load.
- Use repeated trials and compare median timings for stability.

Technology & Dashboard

Implementation plan

PYTHON

Algorithms, dataset generation, benchmarking, and validation.

STREAMLIT

Interactive controls, traces, tables, and results.

PLOTLY

Runtime charts, scaling graphs, and operation comparisons.

Suggested project structure

algorithms/ datasets/ benchmark.py dashboard.py outputs/

Keep algorithm logic separate from the Streamlit interface.

CONCLUSION

Algorithm Race Dashboard

A practical DAA prototype combining repeatable datasets, algorithm traces, performance measurements, and interactive visualization.

- Compare sorting algorithms fairly.
- Visualize Binary Search step by step.
- Connect measured performance with theoretical complexity.

GENERATE

COMPARE

VISUALIZE

UNDERSTAND